

Introduction

The purpose of this project is to explore different linear data structures and some examples of real-world applications utilizing them, and strengthen the students' familiarity with object-oriented programming.

- **Implement:** Using what was learned in lecture, implement a Deque using a Circular Array, given the Deque interface.
- **Refactor:** Learn how to refactor data structures previously implemented (SinglyLinkedList) to add new functionality to them.
- **Apply:** Learn how to limit yourself to using user-facing methods to solve themed problems involving Deques and Linked Lists.

In this project, you will implement an ArrayDeque by filling in a shell file we provide, similar to Homework 1. Then, you will implement a specialized LinkedList with an Iterator, utilizing your SinglyLinkedList from Homework 1. Finally, you will apply these data structures to create a simple music player called Quackify.

Feel free to ask questions on Piazza or come to Office Hours if you need any help or clarifications.

If you complete this project **by yourself**, you will learn a lot and build your competency in solving difficult problems with code, which will be extremely useful for future coursework, passion projects, and job interviews. Thus, we require that you do not use generative AI to write the code, nor can you copy code from other students.

Provided Files

1. **Main.java**
The main class for your project. Here, you will instantiate the classes you create, which is necessary to test your code.
2. **ArrayDeque.java**
Shell class for you to complete to implement an ArrayDeque.
3. **StaticEndlessLinkedList.java**
An interface for you to extend and implement an endless Linked List with.
4. **StaticQuackify.java**
An interface for you to extend and implement several methods for our music player, Quackify.
5. **DequeStudentTest.java**
Part 1 local testing file (non-comprehensive).
6. **EndlessStudentTest.java**
Part 2 local testing file (non-comprehensive).
7. **QuackifyStudentTest.java**
Part 3 local testing file (non-comprehensive).
8. **QuackifyUI.java**
Part 3 local testing file with a User Interface (UI) to test your backend visually!

Deliverables

You will need to create your own files that implement the interfaces we provide. It does not matter what you name the files as long as you follow the directions for inputs/outputs in `Main.java`.

You must submit **all** files that you create, along with your modified `Main.java` and `ArrayDeque.java`. To submit your code, follow the process below depending on your operating system:

Windows: With your project open in File Explorer, double click on `submit-windows.bat`

MacOS/Unix: With your project open in your IntelliJ, open the integrated terminal (the [>_] icon). Run the following command: `./submit-unix.sh`

These will create a file called `submission.zip` inside your project folder, and this is what you should upload to Gradescope.

Once submitted, double-check that it has uploaded properly on Gradescope. It will tell you if your code failed to compile.

Getting Started

1. **Download the files:** navigate to the Canvas assignment, and download and unzip the folder.
2. **Setup the project:** if you are using IntelliJ, open the project in the root directory of the unzipped project files. The 'src' directory should already be marked as sources root (blue) the 'tst' directory should already be marked as test sources root (green). *Optional tip: right-click on the icon next to the top line of any Javadoc and select 'Render All Doc Comments' to view the Javadocs in a user-friendly format.*
3. **Complete parts 1 and 2 first:** these parts are hard prerequisites to part 3. Any classes you create in these parts must be located in their respective packages.
4. **Complete part 3:** use the data structures from 1 and 2 to solve the scenario in part 3. Any classes you create in this part must be located in the respective package.
5. **Test locally:** ensure you fill out the `Main.java` file with your instances from parts 2 and 3. From there, you can run the student test files located in `tst`. Note that these files are not comprehensive, and you are encouraged to write your own JUnits to test edge cases.
6. **Submit to Gradescope:** follow the above section's steps to zip your files and upload that to Gradescope, which will run the full suite of tests.

Part 1: ArrayDeque

Background

A **Doubly-Ended Queue**, commonly known as a **Deque**, is an abstract data type (ADT) that allows adding and removing from two ends: the front and the back. Think of it like a normal Queue, where you add to one end and remove from the other, but instead, with a Deque, you can operate in both directions.

A Deque can be backed by either a LinkedList or an array structure. For this project, we will focus on an **Array-backed Deque** implementation that utilizes a **Circular Array**. However, you should still be familiar with how a LinkedList-backed Deque functions conceptually, as is taught in lecture.

Anything done with a Deque can also be achieved with some combination of Stacks and Queues, and vice versa. In the real world, engineers tend to prefer using Deques, since using one data structure instead of multiple usually yields simpler code. You will inevitably encounter situations where the humble Deque is the best tool at your disposal for a coding interview or personal project.

Implementation

The backing array in your `ArrayDeque` implementation **must behave circularly**. This means your front variable should wraparound to the beginning and end of the array as you add and remove to take advantage of empty space while maintaining $O(1)$ efficiency for all operations.

Near the bottom of the `ArrayDeque.java` file, there is a helper method called `mod(int index, int modulo)` to help you with the math part of this circular logic. This `mod` function is similar to the normal mod operator (%), but handles negative numbers as a Circular Array would expect (if the regular mod returns a negative, adds the result by the divisor).

Your front variable in `ArrayDeque` should represent the index holding the first element of the Deque. **Failure to follow this convention will result in loss of points.**

To access the back of the Deque, you can add the size to the front variable to get the index of the back of the Deque, though you will have to account for the circular behavior yourself. *Hint: you may find the `mod` method useful here.*

Do not shift any elements during a remove. This also means that if there are empty spaces at the front of the array when adding, the back of the Deque should wrap around to the front of the array and make use of those spaces.

Additionally, after removing the last element in the Deque, move the front variable like you normally would. Do **not** explicitly reset it to 0.

Resizing

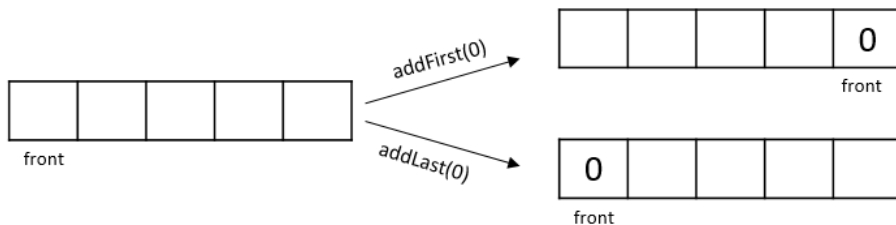
The starting capacity of the `ArrayDeque` should be the constant `INITIAL_CAPACITY` defined in `ArrayDeque.java`. If, while adding an element, the `ArrayDeque` does not have enough space, you should resize the backing array to **twice** its old capacity.

When resizing the backing array, “unwrap” the data. Realign the Deque with the front of the new array during the transfer. The front variable of the Deque is once again at index 0.

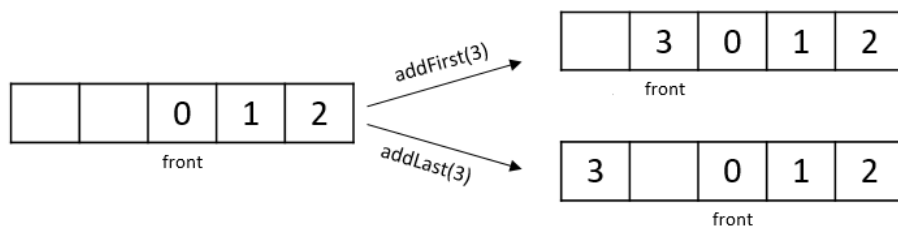
Examples

The examples below demonstrate what the Deque should look like at various states.

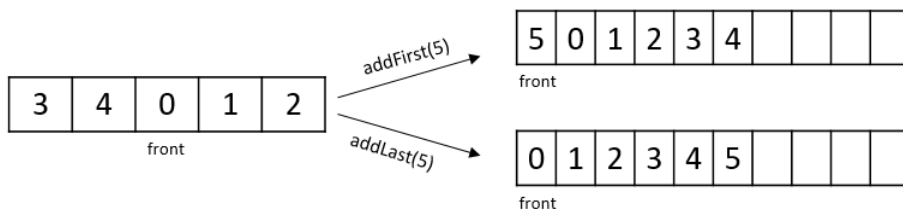
In the example below, the Deque begins empty (initial state). When adding to the front, the front wraps around to the back. The element ends up at the end of the array, since adding to the front adds to the index before the front variable.



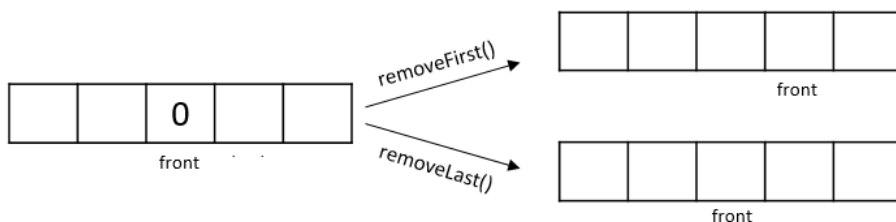
In the example below, adding to the back causes the newly added element to wrap around to the front.



In the example below, adding another element causes the Deque to resize. The array capacity is doubled and the front element moves to index 0.



In the example below, the last element of the Deque is removed, but front moves as expected. The front variable is not explicitly set to 0.



Play around with [CSVistool.com](https://www.csvistool.com) to see more examples and verify your understanding.

Local Testing

To test your `ArrayDeque` locally, you can run the `DequeStudentTest.java` file, which contains several JUnits to test basics functionality. These tests are **not comprehensive**, meaning they do not cover all edge cases. You should write more tests yourself if you want to verify that edge cases are handled by your code as you expect. Note that one of the constructor tests requires completing Part 2. You will need to complete the relevant method in `Main.java` before testing.

Part 2: EndlessLinkedList

Background

In Homework 1, you implemented a `SinglyLinkedList` without a tail. Now, we want to create a new type of `LinkedList` that can iterate endlessly!

For some context, in Part 3, we will be making a music player that contains a playlist of songs. A playlist is a list of songs that is circular in nature. When you finish playing the last song in the playlist, the next song to play will be the one at the top.

To make the Linked List *endless*, you will implement an `Iterator` that returns the head of the list when the end is reached. Thus, when the user iterates through the list and reaches the end, they will seamlessly be brought back to the beginning. **This is not the same as a `CircularSinglyLinkedList`** (although you could implement this using one).

Implementation

To think about how to approach implementing the `EndlessLinkedList`, consider your `SinglyLinkedList` code from Homework 1. One important design choice to consider is: **Should there be a tail pointer?** Think about what you need to do to achieve the required time complexities.

The methods you will need to implement are in **`StaticEndlessLinkedList.java`**:

- **Constructor** → creates a new instance of your `EndlessLinkedList` object
- **`addFirst(data)`** → adds a data to the front of the list in $O(1)$ time
- **`addLast(data)`** → adds a data to the back of the list in $O(1)$ time
- **`removeFirst()`** → removes a data from the front of the list in $O(1)$ time
- **`removeLast()`** → removes a data from the back of the list in $O(n)$ time
- **`removeValue(data)`** → removes a provided data value from the list in $O(n)$ time
- **`get(index)`** → returns the data at the provided index in $O(n)$ time. **Used in most test cases.**
- **`reverse()`** → reverses the order of the data in the list in $O(n)$ time
- **`iterator()`** → returns an `Iterator` that can iterate through the list and wraps to the front

To begin Part 2, create a new Java class that *extends* the `StaticEndlessLinkedList` abstract class. If you do not remember how abstract classes work, check out [this documentation](#). Recall that to extend an abstract class, you must write code for all of the abstract methods using the method headers listed.

Once you have made your new class, write out all of the method headers from the abstract class so that the compiler is satisfied. Then, begin implementing each method! This is where you will want to reference your `SinglyLinkedList` from Homework 1. Most of the methods are similar to what you have already written.

If an input parameter is null or out of bounds, throw an `IllegalArgumentException`. For example, if the `data` passed in to `addFront(data)` is null, throw this exception. Similarly, if data in `removeValue(data)` is not found in the list, throw a `NoSuchElementException`. Write descriptive error messages!

Iterator

In addition to all of the functional methods, you will also have to implement `Iterable`. This means that you will need to create and return an `Iterator` that allows you to iterate through the `LinkedList` (i.e., through a while or for-each loop).

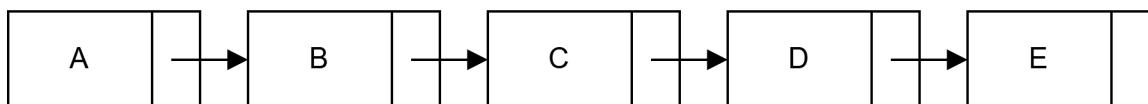
You must also make it so the `LinkedList` **wraps around to the front** when you reach the end. In other words, when you iterate past the last element in the list, the next element to be accessed should be the head. This way, the iterator never ends!

Refer to the Java documentation on iterators [here](#). There is also a helpful refresher module on Canvas called **Module 0 - Java Review** that includes information and examples.

For this project, we recommend creating a private inner class that implements `Iterator`. The class should contain a constructor, a `hasNext` method, and a `next` method. Consider what class variable would be useful to initialize.

If there is nothing in the list, the iterator should throw a `NoSuchElementException` when `.next()` is attempted.

In the example below, calling the iterator's `.next()` method repeatedly should return the following values in order: **A, B, C, D, E, A, B, C, ...**



Once you have implemented the iterator, you can go back to `ArrayDeque.java` to complete the second constructor that takes in a `StaticEndlessLinkedList` parameter, if you have not already done so. This `ArrayDeque` constructor should iterate through all of the data in the `EndlessLinkedList` and push it into a new backing array, in order.

Local Testing

To test your `EndlessLinkedList` locally, you can run the `EndlessStudentTest.java` file, which contains several JUnits to test basic functionality. These tests are **not comprehensive**, meaning they do not cover all edge cases. You should write more tests yourself if you want to verify that edge cases are handled by your code as you expect. You will need to complete the relevant method in `Main.java` before testing.

Part 3: Quackify

You must use only your own two data structures (ArrayDeque, EndlessLinkedList), and no other data structures, including normal arrays, in this part.

Background

In order to finance our rubber duck obsession, CS 1332 is starting a new business venture: Quackify, a Spotify competitor! Quackify is a music app that lets you add and remove songs from a playlist, play the songs in the playlist, and more! We need your help building it.

The required methods for Quackify are listed in `StaticQuackify.java`, which you should implement in your own class:

- `Quackify()` → a constructor that creates a new instance of your Quackify object
- `Quackify(Random)` → overload your constructor with one that takes in a Random object
- `play()` → set the state of Quackify to be playing a song
- `stop()` → set the state of Quackify to **not** be playing a song
- `isPlaying()` → returns whether a song is currently playing
- `getPlaylist()` → return the EndlessLinkedList representing the playlist
- `size()` → return the size of the playlist
- `currentSong()` → return the current song being played
- `nextSong()` → skip to the next song and return it
- `addSong(song)` → add a song to the top of the playlist
- `removeSong(song)` → remove a song from the playlist
- `reverse()` → reverse the order of the songs in the playlist
- `randomSong()` → play a random song from the playlist and return it
- `isPalindrome()` → return true if the playlist is a palindrome of songs
- `undo()` → undo the most recent playlist operation (add or remove)
- `redo()` → redo the most recent playlist operation that was undone

See the below sections for more information on each method. Note that unlike for Part 2, we provide a static **interface** instead of an abstract class. Therefore, you need to use the `implements` keyword instead of `extends`. Refer to the [Java interface documentation](#) as needed.

To simplify the code, songs should just be represented as Strings. In other words, rather than storing a bunch of custom "Song" objects, you can simply store Strings. **Song names are CASE-SENSITIVE for all methods, where relevant.**

For methods that require your ArrayDeque implementation, make sure to import `implement.ArrayDeque`, not `java.util.ArrayDeque`, otherwise you may lose a significant amount of points. Similarly, you should import `refactor.EndlessLinkedList` to access your EndlessLinkedList implementation.

Local Testing

We have provided basic unit tests for you to verify core functionality with. To run these tests, you can execute the `QuackifyStudentTest.java` file in the 'tst' folder. You will need to complete the relevant method in `Main.java` before testing.

For a more fun and interactive test, we've also provided a frontend User Interface (UI) that uses your Quackify code! You can use this app to test all of the functionalities you implemented. To start the app, run the `QuackifyUI.java` file in the 'tst' folder.

We recommend testing incrementally. Code one or a few methods, and then test it with the UI before moving on.

Part 3a: Basic Methods

1. play

This method should set some flag within Quackify to say that songs are currently being played. This is important because a user should *not* be able to add or remove songs from the playlist when the music is playing. Also, to start music playback, **instantiate your iterator**. You should always start by playing the **most recently added** song in the playlist. This should be in $O(1)$ time.

Throw an `IllegalStateException` if the playlist is empty, since there are no songs to play. Also throw the exception if the music is already playing.

2. stop

Similar to `play()`, but now we should set the flag to indicate music is *not* playing. Playlist operations (add and remove songs) are now allowed. Stop music playback by setting the iterator to `null`. This should be in $O(1)$ time.

Throw an `IllegalStateException` if the playlist is empty, since there are no songs to stop. Also throw the exception if the music is already stopped.

3. isPlaying

In $O(1)$ time, return true if Quackify is currently playing a song, and false if not.

4. getPlaylist

Return the playlist as an `EndlessLinkedList`. Return an empty `EndlessLinkedList` if the playlist is empty. This should be in $O(1)$ time.

5. size

In $O(1)$ time, return the current size of the playlist (i.e. the number of songs in the playlist).

6. currentSong

In $O(1)$ time, return the name of the song that is currently playing. If no song is playing, throw an `IllegalStateException`.

7. nextSong

Using the iterator, skip to the next song in the playlist. If no song is playing or the playlist is empty, throw an `IllegalStateException`. If you reach the last song in the playlist, go back to the beginning. This should be in $O(1)$ time. *Hint: Think about how you implemented your `EndlessLinkedList` in Part 2 – you already did the work for this.*

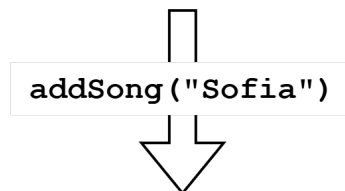
Note that the first song to play should be the song that was most recently added, and the last song to play is the oldest song in the playlist.

8. addSong

Given a song name as a string, add the song to the playlist. In the list, recently-added songs should come before older songs. In other words, the first song to be added should be at the bottom of the list, and the newest song (being added by this method) should be at the top.

This should be in $O(1)$ time. Note that you cannot add a song if music is currently playing; in this case, throw an `IllegalStateException`. If the input song name is null or empty, throw an `IllegalArgumentException`.

[Bags, Amoeba, North, Alewife]



[Sofia, Bags, Amoeba, North, Alewife]

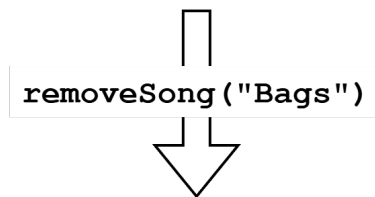
9. removeSong

Given a song name as a string, remove the **first occurrence** from the playlist. The song could be located anywhere in the playlist. This should be in $O(n)$ time.

Note that you cannot remove a song if music is currently playing; in this case, throw an `IllegalStateException`. If the input song name is `null` or empty, throw an `IllegalArgumentException`. If the input song does not exist in the playlist, throw a `NoSuchElementException`.

In the example below, the user really likes the song *Bags*, so it appears several times in their playlist. Calling `removeSong("Bags")` deletes only the first instance of *Bags*.

[Sofia, Bags, North, Bags, Bags]



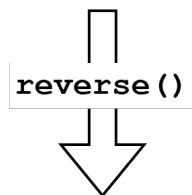
[Sofia, North, Bags, Bags]

10. reverse

Reverse the order of all of the songs in the playlist. That is, the song currently at the back of the playlist should now be at the front, and all other songs should similarly move into reverse order.

You must implement this method in $O(n)$ time and in-place, without using any auxiliary space. If the playlist is empty or music is playing, throw an `IllegalStateException`.

[Sofia, Bags, North, Alewife, Bags]



[Bags, Alewife, North, Bags, Sofia]

Part 3b: Challenge Methods

11. randomSong

Using the iterator, play a random song from the playlist, and return the name of the song that is chosen. To do this, you should generate a random integer using the bounds $[0, n)$. The randomly-chosen integer is the index of the song that should be played.

This feature only works when the user is actively listening to music. Throw an `IllegalStateException` if the music is currently stopped. If this `Quackify` instance was created without a provided `Random` object, you should instantiate a new `Random` object. This method should work in $O(n)$ time. *Hint: You must use `Random.nextInt(b)` to pick a random number. You must use the one-argument method, where b is the exclusive upper bound.*

In the example below, the index 3 is randomly chosen, thus *Alewife* begins playing.

0	1	2	3	4
[Bags,	Amoeba,	North,	Alewife,	Sofia]

12. isPalindrome

As an Easter egg, want to display a fancy confetti effect if the playlist is a palindrome. To do that, we need a way to determine if the playlist is a palindrome. A palindrome is a word that is read the same both forwards and backwards, such as “racecar” or “ABBA”. In this context, you can consider each song name analogous to a character, so the first and last song names must match exactly, and so on.

Return true if the playlist is currently a palindrome of songs. Throw an `IllegalStateException` if the playlist is empty. This method should work in $O(n)$ time. A playlist of size 1 is a palindrome.

You must use your Deque implementation for this method!

⇒ Make sure to import `implement.ArrayDeque`, not `java.util.ArrayDeque`.

To help illustrate what a palindromic playlist looks like, here are some examples:

Palindrome: [Bags, Amoeba, North, Amoeba, Bags]

Palindrome: [Sofia, Sofia, Sofia, Sofia]

NOT Palindrome: [Bags, Amoeba, North, Bags]

13. undo

Whenever you add or remove from a playlist, we consider that to be an “operation” on the playlist. Users of Quackify may accidentally add a song they don’t like to their playlist, or remove their favorite song, so we must provide an easy way to undo blunders like these.

To undo an **add** operation, you need to remove the song you just added. There are two components to consider here: how do we know which song was just added, and how do we remove it?

To undo a **remove** operation, you need to add the removed song back into the playlist **at its original spot**. While a user can only add to the end of the playlist, a user can remove from *any* position, which makes it non-trivial to add it back at that same spot. Consider what information you need to keep track of when a user removes a song, and how you can store that information.

To undo a **reverse** operation, simply un-reverse the list. Don’t overthink this.

You need to handle sequential undo operations. For example, if you just added several songs, hitting the undo button three times should remove the last three added songs. Thus, it is not sufficient to simply save the last song that was added or removed. You must save *all* of them, in order. What data structure could be useful for this?

This method should run in $O(n)$ time. Throw an `IllegalStateException` if the music is playing or there is nothing to undo. See examples on the next page.

You must use your Deque implementation for this method!

14. redo

Users of Quackify may undo an operation, but then realize that it was not an accident and want to redo it (or, in other words, undo the undo).

To redo an undone **add** operation, you need to add back the song that was originally added and removed.

To redo an undone **remove** operation, you need to again remove the song that was added back into the playlist **at its original spot**, after being initially removed.

To redo a **reverse** operation, simply re-reverse the list. Don’t overthink this.

You need to handle sequential redo operations. For example, if you just added several songs, hitting the undo button three times should remove the last three added songs, and then hitting the redo button three times should add those three songs back. **If the user does a new operation after an undo, they should no longer be able to redo the undone action (the history is lost).**

This method should run in $O(n)$ time. Throw an `IllegalStateException` if the music is playing or there is nothing to redo. See examples on the next page.

You must use your Deque implementation for this method!

Undo and Redo Examples

The initial state of the playlist is below. The result of each operation is shown to the right of the method call.

<i>Initial State</i>	[Bags, Amoeba, North, Alewife, Sofia]
<code>removeSong("North")</code>	[Bags, Amoeba, Alewife, Sofia]
<code>addSong("Juna")</code>	[Juna, Bags, Amoeba, Alewife, Sofia]
<code>undo()</code>	[Bags, Amoeba, Alewife, Sofia]
<code>undo()</code>	[Bags, Amoeba, North, Alewife, Sofia]
<code>redo()</code>	[Bags, Amoeba, Alewife, Sofia]
<code>reverse()</code>	[Sofia, Alewife, Amoeba, Bags]
<code>undo()</code>	[Bags, Amoeba, Alewife, Sofia]
<code>addSong("Sinking")</code>	[Sinking, Bags, Amoeba, Alewife, Sofia]
<code>redo()</code>	IllegalStateException

The last `redo()` call results in an `IllegalStateException` because the redo history is lost when you do a new operation. You can only redo *immediately* after an undo or redo.

Grading

Here is the grading breakdown for this project. There are various deductions not listed that are incurred when breaking the rules listed in this document, including inefficient code.

Part	Component	
1	ArrayDeque	10%
2	EndlessLinkedList	15%
3	Quackify	25%
All	Demo	50%

Autograder

The autograder for this course will give you some helpful information, but it will be limited. Every time you submit, you will receive info about any compiler errors, Checkstyle deductions, and general formatting issues. The **first three times per day** you submit, you will see your per-method score. You will not see the number of failed tests or any information about the tests themselves.

Note that while this score is generally representative, **it is not final**. We go over each submission manually, and it is possible for autograder points earned to be deducted in certain cases, including, but not limited to, attempting to brute-force test cases. Additionally, we will manually grade efficiency, so that score is not included in the autograder score. Finally, **we reserve the right to update our test cases** should any defects be discovered, which may result in slight changes to your autograder score.

Checkstyle

Your code must comply with all of the Checkstyle requirements. You will lose a half-point for each Checkstyle issue, capped at -5.0 points. Gradescope will tell you if there are Checkstyle issues in your submission, and you can also manually run Checkstyle by downloading it from the [Course Website](#).

Collaboration Policy

Every student is expected to read, understand, and abide by the [Georgia Tech Academic Honor Code](#).

When working on assignments, you **may not** directly copy code from any source (other than your own past submissions). You are welcome to collaborate with peers and consult external resources, but you **must** personally write all of the code you submit. **You must list, at the top of each file in your submission, every student with whom you collaborated and every resource you consulted while completing the assignment.**

You may not directly share any files containing assignment code with other students or post your code publicly online. If you wish to store your code online in a personal private repository, you can use [GitHub Enterprise](#) to do this for free.